Quick Setup Info for OrangeHRM:

URL: https://opensource-demo.orangehrmlive.com/

Username: `Admin`

Password: `admin123` Exercise

1: Basic POM (Login) Target: The landing page. Elements: Use the `username` and `password` input fields and the `Login` button. Goal: Your `LoginPage` class should hide the `By.name("username")` or `By.xpath(...)` locators from the actual `@Test` script.

File 1: login_page.py

```
from selenium.webdriver.common.by import By

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        # Locators
        self.username_field = (By.NAME, "username")
        self.password_field = (By.NAME, "password")
        self.login_button = (By.XPATH, "//button[@type='submit']")

    # Enter username
    def enter_username(self, username):
        self.driver.find_element(*self.username_field).send_keys(username)

    # Enter password
    def enter_password(self, password):
        self.driver.find_element(*self.password_field).send_keys(password)
```

```
    # Click login button
    def click_login(self):
        self.driver.find_element(*self.login_button).click()

    # Complete login method
    def login(self, username, password):
        self.enter_username(username)
        self.enter_password(password)
        self.click_login()
```

Exercise 2: Page Factory & Synchronization Target: The Login process and the Dashboard. Elements: Use `@FindBy(name = "username")` etc. Synchronization: After clicking login, the page often takes a second to load the Dashboard. In your Page Object, use `WebDriverWait` to wait for the Dashboard Heading (`

` or `` containing "Dashboard") to be visible before returning control to the test.

File 1: login_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class LoginPage:
    def __init__(self, driver):
```

```
    self.driver = driver
    # Locators
    self.username_field = (By.NAME, "username")
    self.password_field = (By.NAME, "password")
    self.login_button = (By.XPATH, "//button[@type='submit']")
    self.dashboard_heading = (
        By.XPATH,
        "//h6[text()='Dashboard']"
    )

# Login method
def login(self, username, password):
    # Enter username
    self.driver.find_element(*self.username_field).send_keys(username)
    # Enter password
    self.driver.find_element(*self.password_field).send_keys(password)
    # Click login
    self.driver.find_element(*self.login_button).click()

    # Explicit Wait for Dashboard
    wait = WebDriverWait(self.driver, 10)
    wait.until(
        EC.visibility_of_element_located(
            self.dashboard_heading
        )
  )
```

File 2: test_dashboard.py

```
from selenium import webdriver
from login_page import LoginPage
import time

# Launch browser
driver = webdriver.Chrome()
# Open OrangeHRM website
driver.get("https://opensource-demo.orangehrmlive.com/")
# Maximize browser
driver.maximize_window()
# Create page object
login_page = LoginPage(driver)

# Perform login
login_page.login("Admin", "admin123")
print("Dashboard loaded successfully")
# Wait to observe
time.sleep(5)

# Close browser
driver.quit()
```

Exercise 3: Multi-Page Navigation & Chaining Target: "PIM" (Personnel Information Management) section. Flow: 1. `PIMPage` (The Employee List): Implement a method `viewEmployeeDetails(name)` that clicks an employee record. 2. Chaining: That method should return `new PersonalDetailsPage(driver)`. Why: This perfectly demonstrates how navigating from a list to a specific profile transitions the "state" of your automation from one Page Object to another.

File 1: personal_details_page.py

```
class PersonalDetailsPage:
    def __init__(self, driver):
        self.driver = driver
    # Get page title
    def get_title(self):
        return self.driver.title
```

File 2: pim_page.py

```
from selenium.webdriver.common.by import By
from personal_details_page import PersonalDetailsPage

class PIMPage:
    def __init__(self, driver):
        self.driver = driver

    # Click employee record
    def viewEmployeeDetails(self, employee_name):
```

```
        employee = self.driver.find_element(
            By.XPATH,
            f"//div[contains(text(),'{employee_name}')]"
        )
        employee.click()
        # Return next page object
        return PersonalDetailsPage(self.driver)
```

File 3: test_pim.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from login_page import LoginPage
from pim_page import PIMPage
import time

# Launch browser
driver = webdriver.Chrome()
# Open OrangeHRM
driver.get("https://opensource-demo.orangehrmlive.com/")
# Maximize browser
driver.maximize_window()
# Login
login_page = LoginPage(driver)
login_page.login("Admin", "admin123")
# Open PIM page
```

```
driver.find_element(By.LINK_TEXT, "PIM").click()
# Wait for page load
time.sleep(3)
# Create PIM page object
pim_page = PIMPage(driver)
# View employee details
personal_page = pim_page.viewEmployeeDetails("Linda")
# Print next page title
print(personal_page.get_title())
# Wait to observe
time.sleep(5)
# Close browser
driver.quit()
```

Exercise 4: Component-Based Architecture Target: The Sidebar Menu (Admin, PIM, Leave, Time, etc.). Implementation: Create a `SideMenuComponent` class. Since this sidebar exists whether you are on the "Admin" page or the "Leave" page, include an instance of `SideMenuComponent` in both the `AdminPage` and `LeavePage` classes. This prevents you from rewriting the locators for the "Logout" or "Search Menu" items on every single page.

File 1: side_menu_component.py

```
from selenium.webdriver.common.by import By

class SideMenuComponent:
```

```
    def __init__(self, driver):
        self.driver = driver
        # Common sidebar locators
        self.admin_menu = (By.LINK_TEXT, "Admin")
        self.pim_menu = (By.LINK_TEXT, "PIM")
        self.leave_menu = (By.LINK_TEXT, "Leave")
        self.time_menu = (By.LINK_TEXT, "Time")
    # Click Admin menu
    def go_to_admin(self):
        self.driver.find_element(*self.admin_menu).click()
    # Click PIM menu
    def go_to_pim(self):
        self.driver.find_element(*self.pim_menu).click()
    # Click Leave menu
    def go_to_leave(self):
        self.driver.find_element(*self.leave_menu).click()
    # Click Time menu
    def go_to_time(self):
        self.driver.find_element(*self.time_menu).click()
```

File 2: admin_page.py

```
from side_menu_component import SideMenuComponent

class AdminPage:
```

```
    def __init__(self, driver):
        self.driver = driver
        # Include sidebar component
        self.side_menu = SideMenuComponent(driver)
```

File 3: leave_page.py

```
from side_menu_component import SideMenuComponent

class LeavePage:
    def __init__(self, driver):
        self.driver = driver
        # Include sidebar component
        self.side_menu = SideMenuComponent(driver)
```

File 4: test_sidebar.py

```
from selenium import webdriver
from login_page import LoginPage
from admin_page import AdminPage
import time

# Launch browser
driver = webdriver.Chrome()
# Open OrangeHRM
driver.get("https://opensource-demo.orangehrmlive.com/")
# Maximize browser
driver.maximize_window()
```

```
# Login
login_page = LoginPage(driver)
login_page.login("Admin", "admin123")
# Create Admin page object
admin_page = AdminPage(driver)
# Use sidebar component
admin_page.side_menu.go_to_pim()
# Wait to observe
time.sleep(5)
# Close browser
driver.quit()
```

Exercise 5: Data-Driven & List Elements Target: The "Admin" -> "User Management" table. Implementation: Navigate to the Admin page where a table of users is displayed. Use `@FindBy` to grab the list of all usernames in the table column. Your method should iterate through these web elements to verify if a specific user (passed from your Data Provider) exists in the system.

File 1: admin_page.py

```
from selenium.webdriver.common.by import By

class AdminPage:
    def __init__(self, driver):
        self.driver = driver
        # Locator for all usernames in table
```

```
        self.usernames = (
            By.XPATH,
            "//div[@role='row']//div[@role='cell'][2]"
        )

    # Verify user exists
    def is_user_present(self, expected_user):
        users = self.driver.find_elements(*self.usernames)
        # Iterate through usernames
        for user in users:
            if user.text == expected_user:
                return True
        return False
```

File 2: test_user_management.py

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from login_page import LoginPage
from admin_page import AdminPage
import time

# Test data (Data Provider)
test_users = ["Admin", "John", "Linda"]
# Launch browser
driver = webdriver.Chrome()
# Open OrangeHRM website
```

```
driver.get("https://opensource-demo.orangehrmlive.com/")
# Maximize browser
driver.maximize_window()
# Login
login_page = LoginPage(driver)
login_page.login("Admin", "admin123")
# Open Admin page
driver.find_element(By.LINK_TEXT, "Admin").click()
# Wait for page load
time.sleep(3)
# Create Admin page object
admin_page = AdminPage(driver)
# Check users from data provider
for username in test_users:
    if admin_page.is_user_present(username):
        print(username, "exists in system")
    else:
        print(username, "not found")
# Wait to observe
time.sleep(5)
# Close browser
driver.quit()
```